Testbericht 4 – Verrundung, Fase, Shell und Profiloperationen

Darstellung in der Browser-UI · 23. Oktober 2026 · Commit 36fb562

Phase C (Playwright)	Phase A/B (Regression)	Vitest	cargo test --workspace	Fehler in fremden Crates
20 / 20 bestanden	**36 / 36**	**155 / 155**	**220 / 220**	**3**

1 Umfang

Geprüft und vervollständigt wurde die Darstellung der Spec-Abschnitte „Verrundung und Fase“, „Boolesche Operationen“ (Parameter `fillet =`/`chamfer =`), „Profiloperationen und Flächen“, „Primitive/2D“ und „Fehler und Warnungen“ sowie zwei offene Punkte aus Phase A/B.

- **fillet/chamfer-Zeile:** Zonen blau halbtransparent, ausgewählte Kanten orange, „n Zonen · m Kanten“ am Zeilenende (vorhanden, jetzt mit Pixelprüfung). **Neu: Cursor auf einer Zone** zeigt nur diese Zone und nur die Kanten des Blocks, die in ihr liegen (in der Ansicht an den Zonen-Meshes geprüft, `inside`: alle Abtastpunkte, `touch`: einer); eine Zone über `translate` ohne Umriss/Koordinatenkreuze der Transformation. Leere Auswahl gelb mit „0 Kanten“, Fehlschlag rot mit roter Kante und Körper ohne Verrundung (Engine), `difference(fillet = r)`, `$fillet`/`$chamfer`, `select = "touch"` und verschachtelte Blöcke geprüft.
- **shell:** wie der Zonenblock; **neu** sind die geöffneten Flächen des Körpers orange gefüllt (Flächen des Körper-Meshes, die vollständig in einer Zone liegen; Anzahl gegen `opened_faces` der Engine geprüft), auch beim Cursor auf einer Zone.
- **Profiloperationen (neu):** `linear_extrude` – Profil als Umriss unten (bei `center` auf −h/2), Höhengriff oben; `rotate_extrude` – Profil in der xz-Ebene, Drehachse als gestrichelte Linie, Winkelgriff auf dem Ring; `loft` – alle Profile als Umrisse an ihrer Lage, Ergebnis halbtransparent; `sweep` – Pfad als Kurve (kubische Interpolation wie im Kernel), Griffe an Literal-Punkten, Profil am Startpunkt senkrecht zur Starttangente; `surface` – Kontrollnetz als Linien mit Griffen, Fläche beidseitig; `thicken` – Fläche als Umriss, Ergebnis voll. Die Griffe (Höhe, Winkel, Pfadpunkte, Kontrollnetz) lieferte cad-eval schon; neu sind ein Test dafür und ihre Darstellung/Prüfung im Browser.
- **2D-Formen allein:** Umriss mit leichter Füllung in der xy-Ebene; die Warnung bei z-Abweichung von Rasterpunkten erscheint gelb am Zeilenende.
- **Neu einpassen:** nach einer deutlichen Größenänderung (Diagonale × 1,6 oder Mitte um mehr als die halbe Diagonale verschoben) passt die Ansicht selbst neu ein, solange der Nutzer die Kamera nicht bewegt (und keinen Griff gezogen) hat; sonst erscheint der Knopf „Neu einpassen“.
- **Achsgebundenes Ziehen** von Punktgriffen: Umschalttaste – Achse des lokalen Systems aus der Mausbewegung; Tasten X/Y/Z – Achse direkt (erneut: frei), auch mitten im Ziehen. Führungslinie in Achsfarbe, Snap nur auf Ziele auf der Achse, die übrigen Komponenten behalten ihren Text.
- **Beispiele (neu, im Beispielmenü):** griffschale.scad (shell), flasche.scad (rotate_extrude + fillet), lueftungsgitter.scad (linear_extrude + Raster, chamfer =), kabelfuehrung.scad (sweep), freiformdeckel.scad (surface + thicken).
- **Entwurfsmodus beim Ziehen (Engine-Stand 58735e2):** Verrundungen, die länger als 25 ms brauchen, setzt die Engine beim Ziehen aus (`NodeReport.blend_suspended`). Die Statuszeile zeigt dann „Verrundung beim Ziehen ausgesetzt (Zeile …)“, am Zeilenende steht derselbe Hinweis; nach dem Loslassen wertet die Engine vollständig aus und der Hinweis verschwindet (C20).

- **Watchdog-Sperre (Plan 7.3):** die UI merkt sich bei jedem `Progress` den Schlüssel von `current`. Beendet der Watchdog den Worker, bekommt der neue Worker vor `SetSource` `{type: "BlockKeys", keys: […]}` mit allen bisher hängenden Schlüsseln (die Liste bleibt über weitere Neustarts erhalten) und dann die letzte Quelle erneut; der gesperrte Knoten meldet „Zeitüberschreitung, Knoten gesperrt bis zur nächsten Änderung“, seine Zeile ist rot (C19, Unit-Test transport.test.ts). Ohne neuen gesperrten Knoten wird die Quelle nicht erneut gesendet, damit dieselbe Eingabe nicht erneut hängt.
- **tools/report/report2-ui.mjs** läuft wieder eigenständig: fehlen Vitest-Ergebnisse oder enthält das gemeinsame playwright.json keine Phase-A-Szenarien (es hält nur den letzten Lauf), führt es diese Tests selbst aus.

Änderungen an Rust-Crates: `cad-session` – Szenenknoten mit `op_info` (aufgelöste Parameter von fillet/chamfer/shell inkl. `select`, zone, linear_extrude, rotate_extrude, loft, sweep-Pfad, surface-Netz, thicken), `blend_suspended` je Knoten und `suspended_blends` in den Statistiken (aus dem Entwurfsmodus der Engine), Protokoll-Typen neu erzeugt; `cad-eval` – beim freien Ziehen werden Literale, deren Wert sich nicht ändert, nicht neu geschrieben (sonst würde das achsgebundene Ziehen z. B. 5.5 zu 6 runden), Tests für Profil-Griffe; `compat-tests` – Erwartungen für die fünf neuen Beispiele (der Test verlangt einen Eintrag je Datei in examples/spec).

2 Testumgebung

Betriebssystem	Linux 6.18.44-fc-v140 (x64)
CPU	Intel(R) Xeon(R) Processor @ 2.80GHz · 4 Kerne
Node.js	v22.22.0
Browser	Chromium 141.0.7390.37 (headless, Playwright 1.56.1)
Grafik	WebGL über SwiftShader (Software-Rendering, keine GPU)
Fenster	1440 × 900 px, Gerätepixelverhältnis 1
Build	Vite 8.3.5 Produktions-Build, Vorschau-Server (vite preview)
Bibliotheken	React 19.3.1, three.js 0.186.1, CodeMirror 6.43.15, Zustand 5.0.16, Vitest 5.0.5
Engine	cad_wasm_bg.wasm 1.20 MiB, smlib.wasm 4.89 MiB (SMLib-Kernel, Emscripten)
Quelltext	Branch worktree-agent-ab88fcc31de0a88e0, Commit 36fb562

Die End-to-End-Tests laufen gegen den Produktions-Build mit echten Maus- und Tastaturereignissen. Geprüft wird über den Testhaken `window.__cad` (Darstellungsplan, gezeichnete Kanten nach Art, Zonen-/Flächenprüfung, Kamera, Griffe, Editortext) und über Pixel: für eine Ebene (z. B. `edge:selected`, `open-faces`, `mode:profile`, `guide:path`) werden die Bildpunkte gezählt, die sich ändern, wenn sie ausgeblendet wird.

3 Szenarien (Playwright, web/e2e/ui-c.spec.ts)

ID	Szenario	Ergebnis	Dauer	Geprüft
C01	fillet-Zeile: Zonen blau, ausgewählte Kanten farbig, Anzahl am Zeilenende	bestanden	4.1 s	• Darstellung: full@12, blue@17, blue@18 • ausgewählte Kanten gezeichnet: 2 • Pixel der Kanten: 1225, der Zonen: 7447 (Differenz mit/ohne Ebene)
C02	Cursor auf einer Zone: nur diese Zone und ihre Kanten	bestanden	3.0 s	• Zone Kugel: 1 von 2 Kanten, nur blue@18 • Zone Rasterlinie: 1 von 2 Kanten, nur blue@17

C03	Leere Auswahl: Zeile gelb, „0 Kanten“	**bestanden**	2.3 s	• Zeilenklasse cm-l-warning • Knotenstatus warning • keine ausgewählten Kanten gezeichnet
C04	Verrundung gescheitert: Zeile rot, Kante rot, Körper ohne Verrundung	**bestanden**	2.7 s	• Knotenstatus error, 1 betroffene Kante • Mesh des fillet-Knotens: 12 Dreiecke (unverrundeter Würfel) • Pixel der roten Kante: 1184
C05	difference(fillet = r) und $fillet am Primitiv	**bestanden**	3.2 s	• difference: 2 Kanten gezeichnet • $fillet: 12 Kanten • $chamfer am Zylinder: 3 Kanten wie gemeldet
C06	select = "touch" und verschachtelte Blöcke	**bestanden**	3.5 s	• touch: 5 Kanten, Zone: 5 von 5 • äußerer Block: 5 Kanten, eine Zone (blue@20) • innerer Block: 2 Kanten
C07	chamfer in allen drei Formen	**bestanden**	2.8 s	• Zonenblock: 4 Kanten • union(chamfer =): Schnittkanten > 0 • $chamfer: 12 Kanten
C08	shell: Zone blau, geöffnete Flächen farbig	**bestanden**	4.1 s	• geöffnete Flächen Ansicht/Engine: 1/1 • Pixel der geöffneten Fläche: 23047
C09	linear_extrude: Profil als Umriss unten, Höhengriff oben	**bestanden**	6.2 s	• Darstellung full@1, profile@2 • Höhengriff über dem Ursprung • Pixel des Profils: 1629
C10	rotate_extrude: Profil in der xz-Ebene, Drehachse, Winkelgriff	**bestanden**	8.4 s	• Darstellung full@1, profile@2 • Achse: Linie x = y = 0 entlang z • Winkelgriff 270 → 180 • Pixel Achse/Profil: 588/2237
C11	loft: alle Profile als Umrisse, Ergebnis halbtransparent	**bestanden**	3.1 s	• translucent@9, profile@10/11/12 • Pixel der Profile: 2527
C12	sweep: Pfad als Kurve, Griffe an Literal-Punkten, Profil am Startpunkt	**bestanden**	6.5 s	• Kurve durch alle 4 Punkte (Abstand < 0,6 mm) • 4 Griffe (Literal) • Pixel Pfad/Profil: 1853/568
C13	surface und thicken: Kontrollnetz mit Griffen, Fläche beidseitig, Fläche als Umriss	**bestanden**	9.0 s	• 8 Netzlinien, 16 Griffe • mittlerer Punkt gezogen • thicken: full@1, profile@2 • bedeckte Pixel der offenen Fläche von oben/unten: 229152/224965
C14	2D-Formen allein: Umriss mit Füllung, z-Abweichung als Warnung	**bestanden**	2.5 s	• Warnung in Zeile 6 (Diagnose und Zeilenklasse) • alle Darstellungen flach • Pixel Flächenfarbe: 310
C15	Ansicht nach deutlicher Größenänderung neu einpassen	**bestanden**	4.0 s	• automatisch: Abstand > 3 × vorher • kleine Änderung: Kamera unverändert • nach Drehen: Knopf, Kamera unverändert • Knopf: eingepasst, Knopf weg
C16	Punktgriff achsgebunden ziehen (Umschalttaste, X/Y/Z)	**bestanden**	20.5 s	• Umschalt: 27, 5.5, 2.25 (nur x geändert) • erst frei, dann Taste Z: 29, 7, 19 (x, y wie beim Drücken von Z)

C17	Neue Beispiele im Menü	bestanden	9.3 s	griffschale.scad: shell mit Zone g.oben, Kabelausgang per difference, Schraubdome mit difference(fillet = 0.5) (Cursor Zeile 12: full@12, blue@14) · flasche.scad: rotate_extrude eines Polygons aus Rasterpunkten, fillet(3) mit zwei Zonen, Teilumlauf 270° (Cursor Zeile 11: full@11, profile@12) · lueftungsgitter.scad: linear_extrude für Rahmen und Schlitze an indizierten Rasterebenen, difference(chamfer = 0.6) (Cursor Zeile 12: full@12, profile@12) · kabelfuehrung.scad: sweep eines U-Profils entlang eines Spline-Pfads aus Rasterpunkten und Literalen, linearer Kanal (Cursor Zeile 16: full@16, profile@17) · freiformdeckel.scad: surface aus einem 4 × 4-Netz (Rasterrand, Wölbung als Literale), thicken(2) (Cursor Zeile 11: full@11)
C19	Watchdog: hängender Knoten wird gesperrt, Zeile rot, Änderung hebt die Sperre auf	bestanden	5.8 s	• Neustarts: 1, gesperrte Schlüssel: 1 • gesperrter Knoten: Zeile 1 (translate) • nach Änderung: 0 Fehler, kein weiterer Neustart
C20	Entwurfsmodus: Verrundung beim Ziehen ausgesetzt, danach berechnet	bestanden	8.7 s	• Statuszeile während des Ziehens: Zeile 1 • Zeilenhinweis: 2 Kanten · Verrundung beim Ziehen ausgesetzt • danach: blend_suspended überall false, Knoten ok
C18	Messwerte: Auswertzeit fillet, chamfer, shell, sweep, loft, surface	bestanden	16.2 s	fillet(r) Zonenblock, 4 Kanten: Kernel 74 ms (Median aus 5), SetSource → Szene 80 ms · difference(fillet = r), 4 Bohrungen: Kernel 262 ms (Median aus 5), SetSource → Szene 270 ms · chamfer, 12 Kanten ($chamfer): Kernel 105 ms (Median aus 5), SetSource → Szene 106 ms · shell(1.6) eines gefasten Quaders ($chamfer = 4): Kernel 217 ms (Median aus 5), SetSource → Szene 223 ms · sweep, Spline-Pfad mit 4 Punkten: Kernel 154 ms (Median aus 5), SetSource → Szene 160 ms · loft, 3 Profile: Kernel 22 ms (Median aus 5), SetSource → Szene 28 ms · thicken(surface) 4 × 4: Kernel 309 ms (Median aus 5), SetSource → Szene 314 ms

Regression im selben Lauf: Phase A (web/e2e/ui.spec.ts) 22 / 22, Phase B (web/e2e/ui-b.spec.ts) 14 / 14 bestanden.

4 Unit- und Rust-Tests

Vitest-Datei	bestanden	Fehlgeschlagen
web/src/editor/decorations.test.ts	7 / 7	–
web/src/editor/editGroup.test.ts	7 / 7	–
web/src/editor/lang.test.ts	47 / 47	–
web/src/engine/transport.test.ts neu	2 / 2	–
web/src/logic/context.test.ts	5 / 5	–
web/src/logic/curves.test.ts neu	5 / 5	–
web/src/logic/display.test.ts erweitert	23 / 23	–
web/src/logic/drag.test.ts erweitert	12 / 12	–
web/src/logic/format.test.ts	5 / 5	–
web/src/logic/grids.test.ts	5 / 5	–
web/src/logic/handles.test.ts	12 / 12	–

web/src/logic/params.test.ts	7 / 7	–
web/src/logic/positions.test.ts	5 / 5	–
web/src/logic/sceneIndex.test.ts	9 / 9	–
web/src/logic/zones.test.ts neu	4 / 4	–

Neu: kubische Pfad-Interpolation, Startrahmen des Profils, Punkt-in-Mesh mit Toleranz, Kanten in einer Zone (inside/touch), geöffnete Flächen, Kriterium für das Neu-Einpassen. Erweitert: Darstellungsplan für Profiloperationen, Zonen und shell (Fixture `profile.scad` aus der echten Engine), achsgebundenes Ziehen.

Rust-Testprogramm (cargo test --workspace)	bestanden	neue/geänderte Tests
tests/cli.rs	7 / 7	–
src/lib.rs	5 / 5	–
tests/draft.rs	3 / 3	–
tests/engine.rs	24 / 24	–
tests/spec_examples.rs	4 / 4	–
tests/expr.rs	25 / 25	–
tests/grids.rs	14 / 14	–
tests/handles.rs	26 / 26	`profile_operation_handles`, `unchanged_literals_keep_their_text`
tests/limits.rs	8 / 8	–
tests/openscad_compat.rs	1 / 1	–
tests/openscad_deviations.rs	10 / 10	–
tests/tree.rs	24 / 24	–
tests/render.rs	4 / 4	–
tests/protocol_ts.rs	1 / 1	–
tests/session.rs	8 / 8	–
tests/watchdog.rs	2 / 2	–
src/lib.rs	16 / 16	–
tests/openscad_compat.rs	1 / 1	–
tests/spec_models.rs	1 / 1	`spec_models_stage1`
tests/tree_bounds.rs	1 / 1	–
tests/determinism.rs	1 / 1	–
tests/handles.rs	1 / 1	–
tests/kernel.rs	33 / 33	–

5 Messwerte

Modell	Kernel / gesamt	Messung

fillet(r) Zonenblock, 4 Kanten	74 ms / 80 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 148, 75, 74, 64, 73 ms)
difference(fillet = r), 4 Bohrungen	262 ms / 270 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 311, 262, 275, 199, 212 ms)
chamfer, 12 Kanten ($chamfer)	105 ms / 106 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 161, 84, 105, 112, 91 ms)
shell(1.6) eines gefasten Quaders ($chamfer = 4)	217 ms / 223 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 225, 252, 217, 211, 186 ms)
sweep, Spline-Pfad mit 4 Punkten	154 ms / 160 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 154, 129, 161, 145, 158 ms)
loft, 3 Profile	22 ms / 28 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 17, 24, 25, 22, 20 ms)
thicken(surface) 4 × 4	309 ms / 314 ms	Kernel-Zeit der vollständigen Szene / SetSource → Szene in der App, Median aus 5 Varianten (Einzelwerte Kernel 294, 241, 324, 309, 362 ms)
Größte Bilddatei im Bericht	148 KiB	JPEG, Qualität 82

Jede Variante hat andere Parameter (kein Cache-Treffer im betroffenen Knoten). „Gesamt“ ist SetSource bis zur vollständigen Szene im Worker einschließlich Sprachauswertung und Tessellierung; die Zeiten stammen aus headless Chromium mit SwiftShader auf 4 Kernen.

6 Bekannte Grenzen und Abweichungen

- **Kanten einer Zone, geöffnete Flächen:** die Ansicht prüft an den tessellierten Zonen-Meshes (Toleranz 1,5 × Sehnenfehler); die Engine wählt mit der exakten Kernel-Klassifikation. In Grenzfällen (Kante genau auf einer gekrümmten Zonenfläche) kann die Ansicht abweichen; im Test stimmen Anzahl und Engine überein.
- **sweep-Pfad:** die Kurve wird in der UI nach demselben Verfahren wie in der SMLib-Fassade interpoliert (kubisch, Sehnenlängen, bei drei Punkten zwei Zusatzpunkte); ändert sich das Verfahren im Kernel, weicht die Anzeige ab. Der Startrahmen folgt sc_sweep (x = globales X in die Normalebene projiziert).
- **Profil von linear_extrude/rotate_extrude:** gezeigt wird das Mesh der direkten Kinder (2D, flach in ihrer Ebene); z-Verschiebungen von 2D-Transformationen gehen dabei wie in der Engine verloren.
- **Zone über translate:** der Cursor auf der Zone zeigt nur Zone und Kanten, nicht das Koordinatenkreuz der Transformation (Spec-Zeile „Cursor auf einer Zone“ geht vor); der Griff am Zonenmittelpunkt bleibt.
- **Neu einpassen:** automatisch nur, solange die Kamera seit dem letzten Einpassen nicht gedreht/verschoben/gezoomt und kein Griff gezogen wurde; „Alles zeigen“ und die Standardansichten stellen diesen Zustand wieder her.
- **Achsgebunden:** Achsen des lokalen Systems des Griffs (bei 2D-Punkten nur x/y); mit Umschalt wartet der Griff, bis 6 px Bewegung die Achse bestimmen. Wird die Achse erst während des Ziehens gewählt, geht die Achslinie durch die aktuelle Lage.
- **Offene Fläche neben Körpern** auf oberster Ebene ist eine implizite Vereinigung und damit ein Fehler („zuerst thicken“) – Spec-konform, im Test getrennt geladen.
- **Watchdog-Sperre:** gesperrt wird der Knoten aus dem letzten `Progress.current` vor dem Hängen; gesperrt ist sein Inhaltsschlüssel, gleiche Knoten an anderer Stelle sind mitgesperrt. Die Liste gilt bis zum Neuladen der Seite; eine Änderung am Knoten ändert den Schlüssel und hebt die Sperre damit auf.
- **Entwurfsmodus:** der Hinweis zeigt den zuletzt gezeichneten Stand; die Schwelle (25 ms) und die Auswahl der ausgesetzten Verrundungen liegen in der Engine.

- Einige Ergebnisse hängen am Kernel: dose_shell.scad ist wegen F1 derzeit rot (Zeile rot, Körper ohne shell); griffschale.scad verwendet deshalb $chamfer = 4 (auf beiden Plattformen ein korrekter Hohlkörper); die Tests C08, C17 und C18 prüfen zusätzlich, dass ein „ok“-Knoten Geometrie hat.

7 Gefundene Fehler in fremden Crates

Nicht behoben (andere Crates als dieser Auftrag); minimale Beispiele zum Nachstellen im Browser oder mit `cad check datei.scad` (cad-cli, nativer Kernel).

F1 shell eines verrundeten Körpers scheitert; Meldung hängt von der Plattform ab

Crate: kernel-smlib / smlib (sc_shell)

```
grid g { z: o = 18; }
shell(1.6) { box([0, 0, 0], [90, 34, 18], $fillet = 3); zone(g.o, d = 1); }
shell(1.6) { box([0, 0, 0], [90, 34, 18], $fillet = 5); zone(g.o, d = 1); }
shell(1.6) { box([0, 0, 0], [90, 34, 18], $fillet = 6); zone(g.o, d = 1); }
// Maße von examples/spec/dose_shell.scad (dort mit grid g { z: o = 30; }):
shell(1.5) { box([0, 0, 0], [50, 35, 30], $fillet = 4); zone(g.o, d = 1); }
```

Mit dem SMLib-Fork (tangency-labs/smlib, capi-tess @ 7eba8aa) scheitert jede dieser shells, im Browser (Wasm) wie nativ (cad check): $fillet = 3 mit „SmApiShellBrepFull failed: SM_ERR_NULL_POINTER (1006)“, $fillet = 5 und 6 mit „wall thickness 1.6 leaves no cavity (SMLib returned no closed solid)“ bzw. „self-overlapping“, die Maße von dose_shell.scad mit „self-overlapping solid at 10 face(s)“ – dose_shell.scad ist deshalb rot. Für dieselbe Eingabe unterscheiden sich die Meldungen zwischen Wasm und nativ ($fillet = 8: Wasm „self-overlapping“, nativ SM_ERR (1001)). Mit dem vorigen Kernel-Stand meldete $fillet = 5 „ok“ bei leerem Ergebnis und $fillet = 6 „ok“ ohne Außenwand; diese stillen Fehler gibt es nicht mehr. Scharfkantige und gefaste Körper ($chamfer) ergeben korrekte Hohlkörper (griffschale.scad, Messung C18).

F2 surface(points) mit 2 × 2 oder 3 × 3 Kontrollpunkten scheitert

Crate: kernel-smlib (nurbs_surface)

```
surface(points = [[[0, 0, 0], [20, 0, 6]], [[0, 20, 6], [20, 20, 0]]]);
```

„surface gescheitert: SmApiCreateSurfaceFromOrderedPoints failed: SM_ERR (1001)“, ebenso für 3 × 3. cad-eval nimmt ab 2 × 2 Punkte an, die Spec nennt kein Minimum; 4 × 4 funktioniert. Vorschlag: Grad an die Punktzahl anpassen (wie interp_curve bei drei Pfadpunkten).

F3 Nahtkanten zählen als eigene Kanten eines Primitivs

Crate: cad-engine ($fillet / $chamfer)

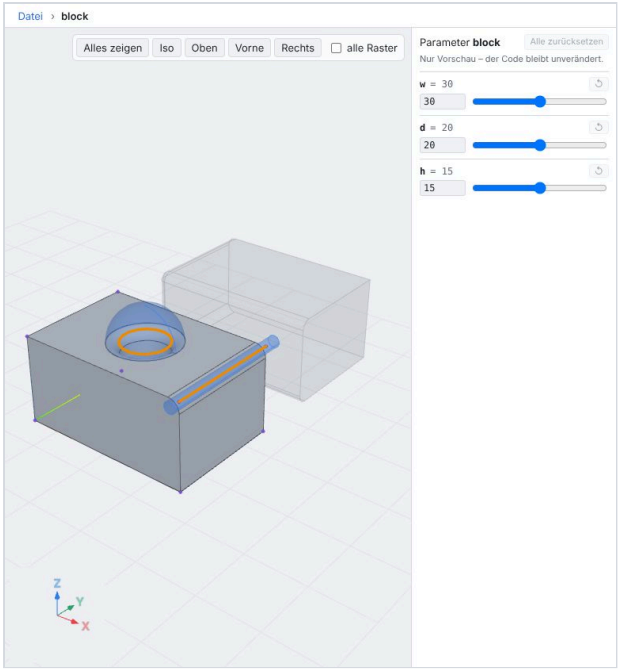
```
cylinder(h = 10, r = 5, $chamfer = 1);   // „3 Kanten“
sphere(5, $fillet = 1);                    // Fehler
```

Beim Zylinder werden Deckel-, Boden- und die Nahtkante ausgewählt („3 Kanten“, gezeichnet 3); bei der Kugel nur die Naht („1 Kante“), und die Verrundung scheitert mit „fillet: none of the edges can be blended (smooth, seam or open edges)“ – Zeile rot statt „0 Kanten“ bzw. ohne Wirkung. Glatte Naht- und Polkanten sollten nicht ausgewählt werden.

8 Screenshots und Codeausschnitte

C01 fillet-Zeile

CURSOR AUF FILLET(2)



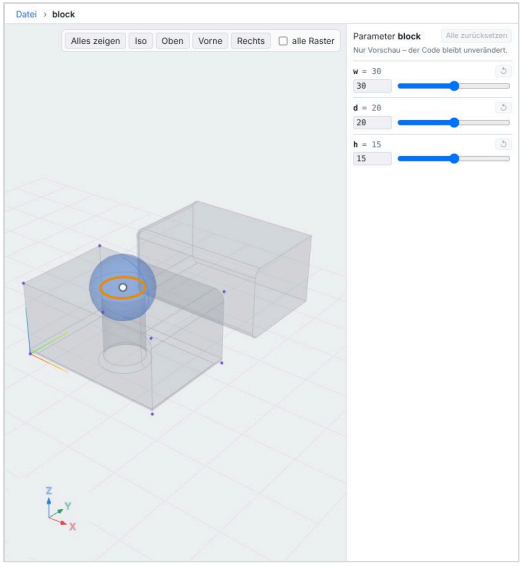
CODE

```
    fillet(1) {
        fillet(2) {
            difference() {
                box(g[links, vorne, boden], g[rechts,
hinten, deckel]);
                translate(p) cylinder(h = h + 1, r =
4);
            }
            zone(g[rechts, *, deckel], d = 2);     //
obere rechte Kante
            translate(p + [0, 0, h]) sphere(6);   //
Bohrungskante oben
        }
        zone(g.boden, d = 1);                      //
alle Bodenkanten (äußerer Radius)
    }
```

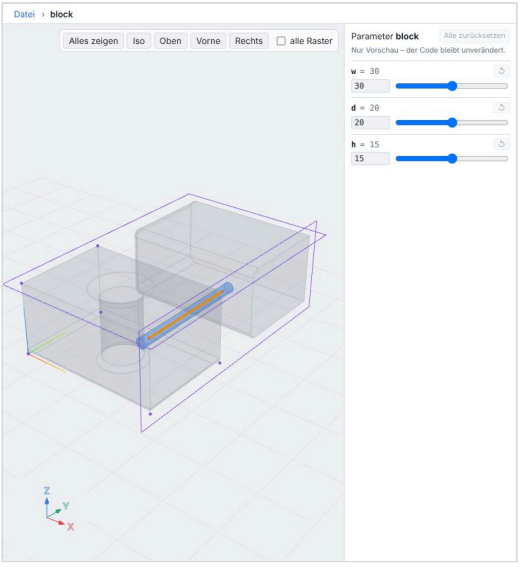
fillet_zonen.scad, Modul block, Cursor auf fillet(2) (Zeile 12): Körper voll, Rest gedimmt; die beiden Zonen (Zylinder um die Rasterlinie g[rechts, *, deckel] und die Kugel über der Bohrung) blau halbtransparent, die zwei ausgewählten Kanten orange; am Zeilenende „2 Zonen · 2 Kanten“.

C02 Cursor auf einer Zone

ZONE KUGEL (ZEILE 18)



ZONE RASTERLINIE (ZEILE 17)



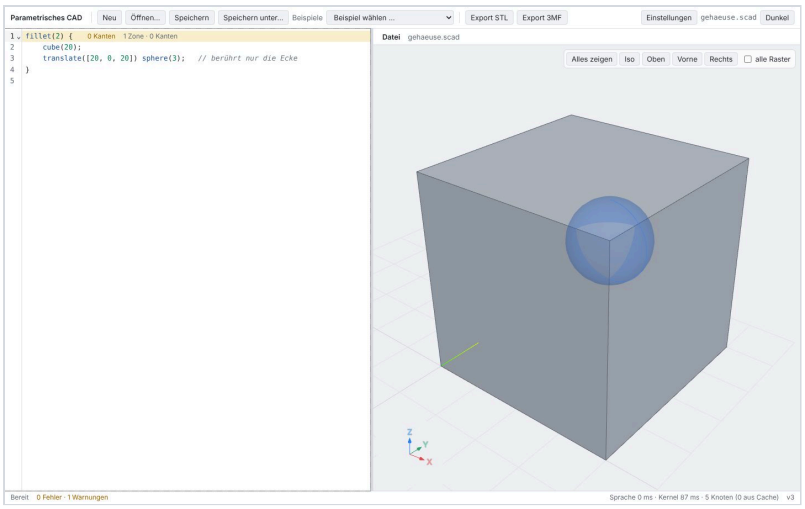
CODE

```
        fillet(2) {
            difference() {
                box(g[links, vorne, boden], g[rechts, hinten, deckel]);
                translate(p) cylinder(h = h + 1, r = 4);
            }
            zone(g[rechts, *, deckel], d = 2);     // obere rechte Kante
            translate(p + [0, 0, h]) sphere(6);   // Bohrungskante oben
        }
```

Cursor auf einer Zone des Blocks fillet(2): nur diese Zone ist blau, von den zwei ausgewählten Kanten des Blocks ist nur die Kante in dieser Zone orange (Kugel: obere Bohrungskante; Rasterlinie: obere rechte Kante). Der Griff am Zonenmittelpunkt (translate) snappt auf Raster (Phase B, B10).

C03 Leere Auswahl

EDITOR UND ANSICHT



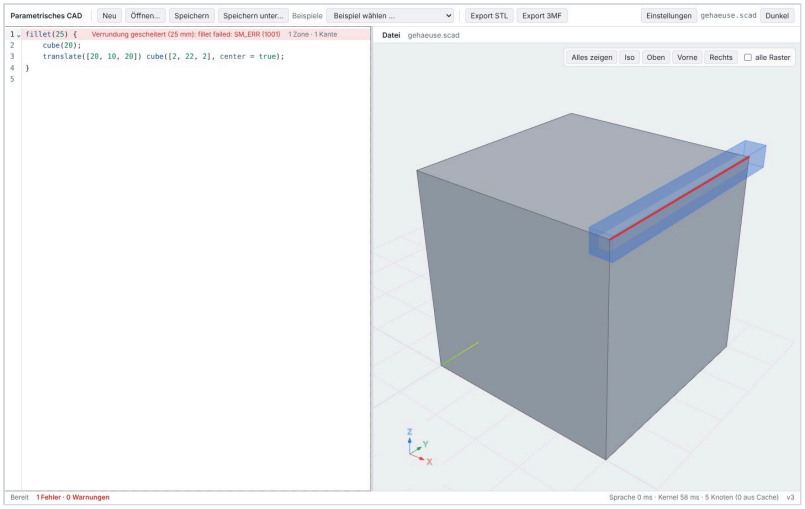
CODE

```
fillet(2) {
    cube(20);
    translate([20, 0, 20]) sphere(3);
// berührt nur die Ecke
}
```

Die Kugel berührt den Würfel nur an der Ecke, keine Kante liegt vollständig in der Zone: Zeile 1 gelb markiert, am Zeilenende „1 Zone · 0 Kanten“ und die Warnung; die Ansicht zeigt den Würfel unverändert (Warnung, kein Fehler).

C04 Verrundung gescheitert

EDITOR UND ANSICHT



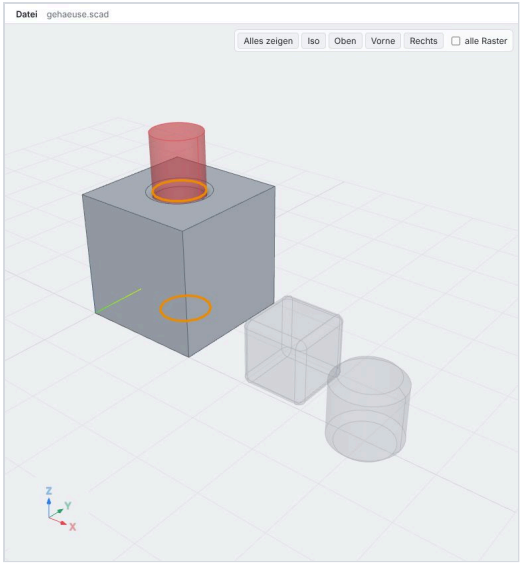
CODE

```
fillet(25) {
    cube(20);
    translate([20, 10, 20]) cube([2, 22,
2], center = true);
}
```

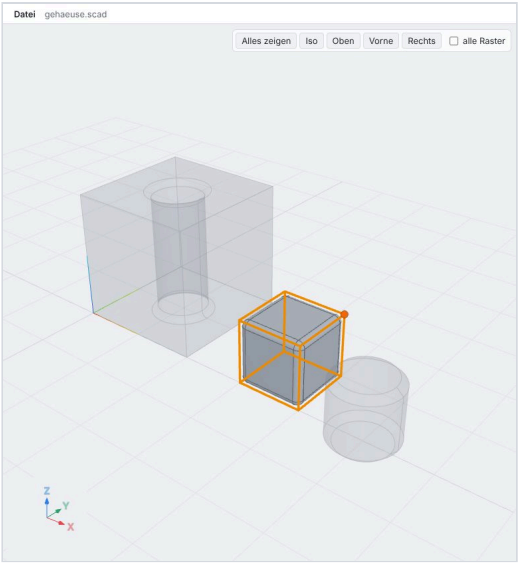
fillet(25) an einer Kante eines 20-mm-Würfels: Zeile 1 rot mit der Kernel-Meldung am Zeilenende, die betroffene Kante rot; die Ansicht zeigt den Würfel ohne Verrundung (Mesh des Knotens = Mesh des Körpers, 12 Dreiecke).

C05 fillet = an Booleans, $fillet/$chamfer an Primitiven

DIFFERENCE(FILLET = 1)



CUBE(10, $FILLET = 1)



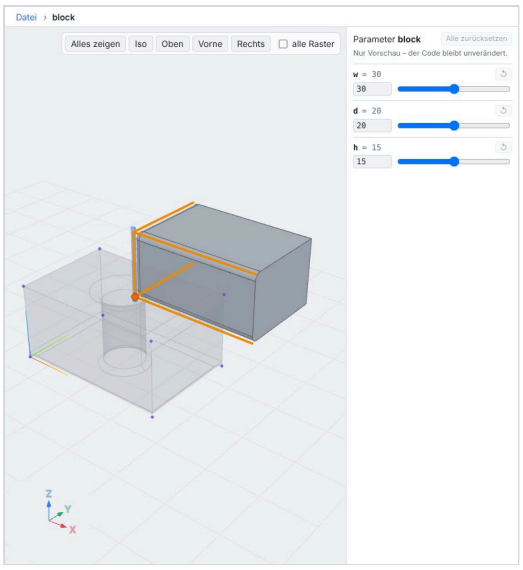
CODE

```
difference(fillet = 1) {
    cube(20);
    translate([10, 10, -1]) cylinder(h = 30, r = 4);
}
translate([30, 0, 0]) cube(10, $fillet = 1);
translate([50, 0, 0]) cylinder(h = 10, r = 5, $chamfer = 1);
```

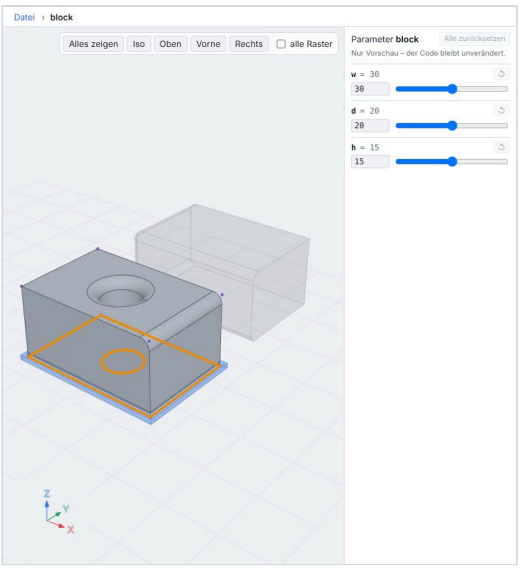
difference(fillet = 1): die zwei durch die Bohrung neu entstandenen Kanten orange, abgezogener Zylinder rot halbtransparent, „2 Kanten“ am Zeilenende. $fillet = 1 am Würfel: alle 12 eigenen Kanten, „12 Kanten“; $chamfer = 1 am Zylinder: die Kanten, die die Engine meldet (3: oben, unten und die Naht – siehe gefundene Fehler).

C06 select = "touch", verschachtelte Blöcke

FILLET(1.5, SELECT = "TOUCH")



ÄUSSERER BLOCK FILLET(1)



CODE

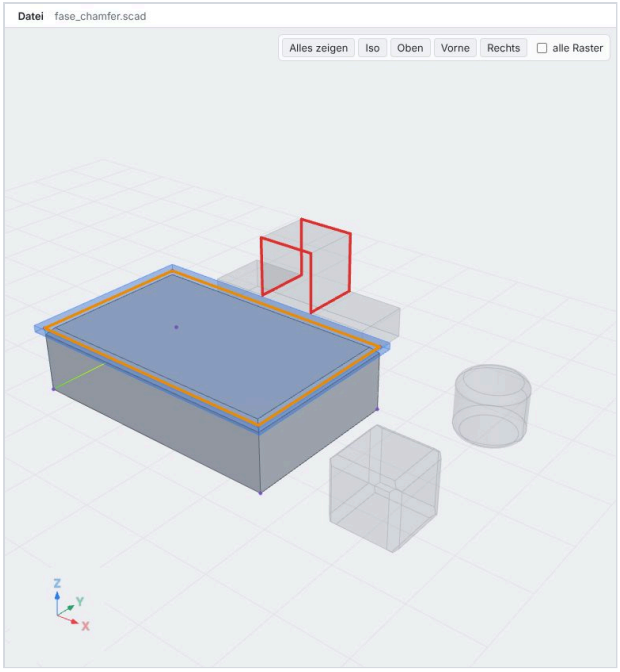
```
    fillet(1) {
        fillet(2) {
            difference() {
                box(g[links, vorne, boden], g[rechts, hinten, deckel]);
                translate(p) cylinder(h = h + 1, r = 4);
            }
            zone(g[rechts, *, deckel], d = 2);     // obere rechte Kante
            translate(p + [0, 0, h]) sphere(6);   // Bohrungskante oben
        }
        zone(g.boden, d = 1);                     // alle Bodenkanten (äußerer Radius)
    }

    translate([0, d + 10, 0]) fillet(1.5, select = "touch") {
        box(g[links, vorne, boden], g[rechts, hinten, deckel]);
        zone(g[links, vorne, *], d = 1);
    }
```

select = "touch": die Zone um die senkrechte Rasterlinie g[links, vorne, *] erfasst die senkrechte Kante und die vier Kanten, die sie an den Enden berühren (auf ganzer Länge), „5 Kanten“; Cursor auf der Zone: alle 5 in dieser Zone. Verschachtelt: der innere Block fillet(2) wählt 2 Kanten, der äußere fillet(1) mit der Zone g.boden die 5 Bodenkanten (vier Außenkanten und die Bohrungskante).

C07 chamfer

CHAMFER(1) MIT ZONE G.O



CODE

```
chamfer(1) {
    box(g[a, v, u], g[b, h, o]);
    zone(g.o, d = 1);
}

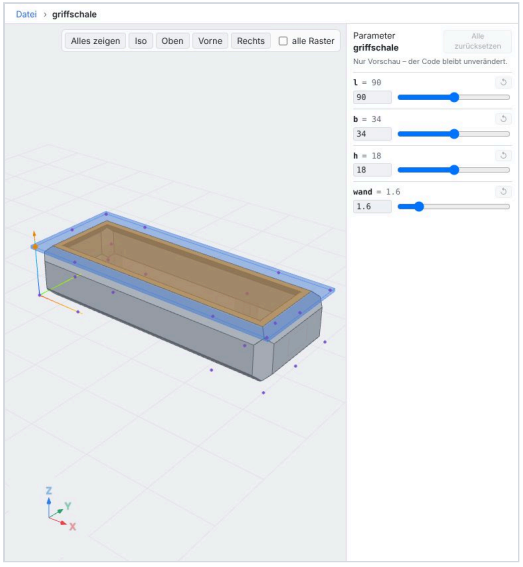
translate([0, 35, 0]) union(chamfer = 0.5) {
    cube([30, 10, 5]);
    translate([10, 0, 0]) cube([10, 10, 15]);
}

translate([50, 0, 0]) cube(10, $chamfer = 1);
```

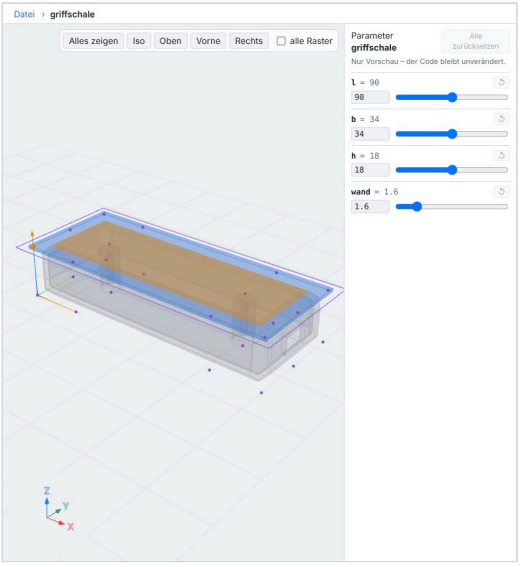
chamfer(1) { box; zone(g.o) }: die vier oberen Kanten orange, Zone blau; union(chamfer = 0.5) markiert die Schnittkanten der beiden Quader, $chamfer = 1 die 12 Würfelkanten.

C08 shell

CURSOR AUF SHELL(WAND)



CURSOR AUF DER ZONE



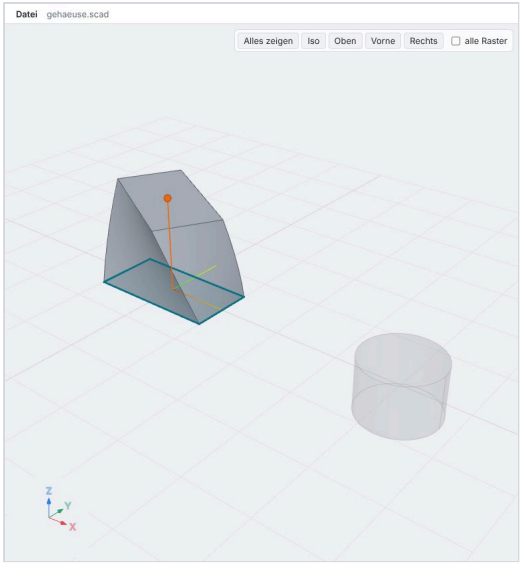
CODE

```
    difference() {
        shell(wand) {
            box(g[hinten, links, boden], g[vorn, rechts, oben], $chamfer = 4);
            zone(g.oben, d = 1);    // Deckelseite offen
        }
        translate(g[vorn, mitte, 9]) cube([8, 10, 6], center = true); // Kabelausgang
    }
```

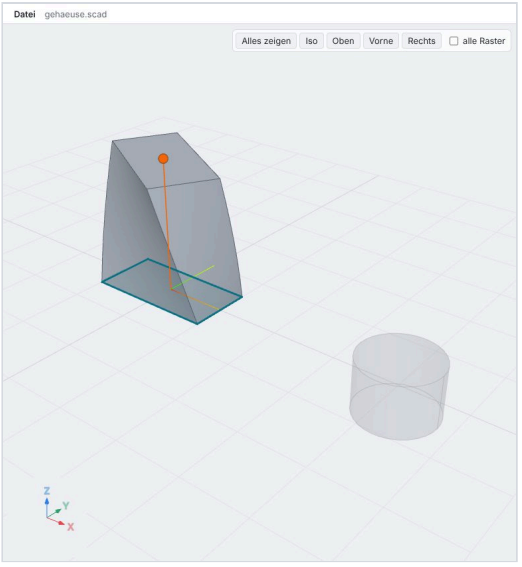
griffschale.scad (neu), Modul griffschale: Cursor auf shell(wand): Hohlkörper voll, Zone (Platte um g.oben) blau, die geöffnete Oberseite des Körpers orange – in der Ansicht aus den Zonen-Meshes bestimmt (1 Fläche, wie die Engine meldet); „1 Zone · 1 Fläche geöffnet“ am Zeilenende. Cursor auf der Zone: nur diese Zone und ihre Fläche.

C09 linear_extrude

VORHER



HÖHE GEZOGEN → 21



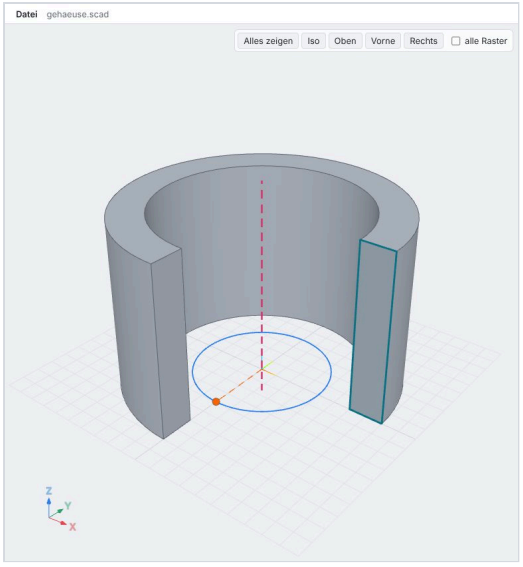
CODE

```
linear_extrude(height = 15, twist = 30)
    translate([-10, -5]) square([20, 10]);
translate([40, 0, 0]) linear_extrude(8, center = true) circle(6);
// nach dem Ziehen:
linear_extrude(height = 21, twist = 30)
```

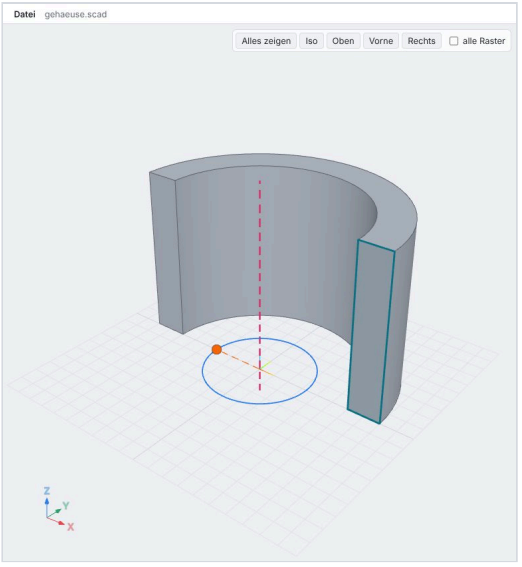
Cursor auf linear_extrude: Körper voll, das Profil (Rechteck) als Umriss in Profilfarbe unten in der xy-Ebene, sichtbar durch den Körper; Höhengriff oben auf der Achse. Griff mit der Maus nach oben gezogen: height = 15 → 21. Bei center = true liegt der Umriss auf −h/2.

C10 rotate_extrude

ANGLE = 270



WINKELGRIFF AUF 180° GEZOGEN



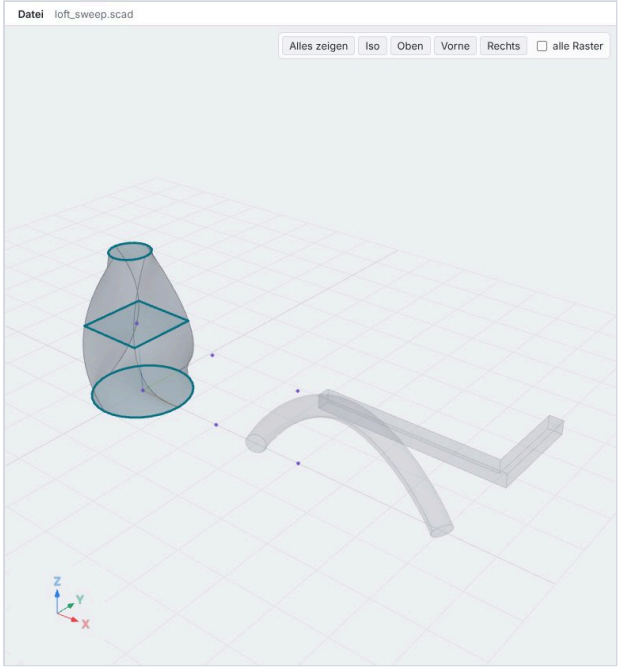
CODE

```
rotate_extrude(angle = 270)
    translate([12, 0]) square([4, 20]);
// nach dem Ziehen:
rotate_extrude(angle = 180)
```

Cursor auf rotate_extrude: Körper voll, das Profil (Rechteck x = 12…16, y = 0…20) als Umriss in der xz-Ebene (Profil-y wird z), die Drehachse z als gestrichelte Linie, der Winkelgriff auf dem Ring um z. Ring mit der Maus auf 180° gezogen: angle = 270 → 180 (Raster 15°).

C11 loft

CURSOR AUF LOFT()



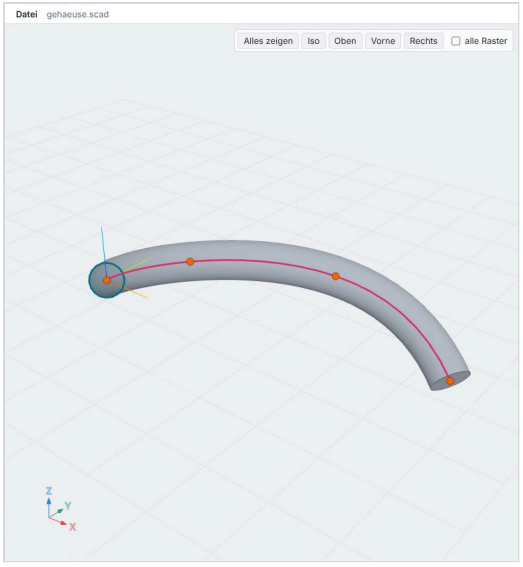
CODE

```
loft() {
    circle(10);
    translate([0, 0, 15]) square(14, center = true);
    translate([0, 0, 30]) circle(4);
}
```

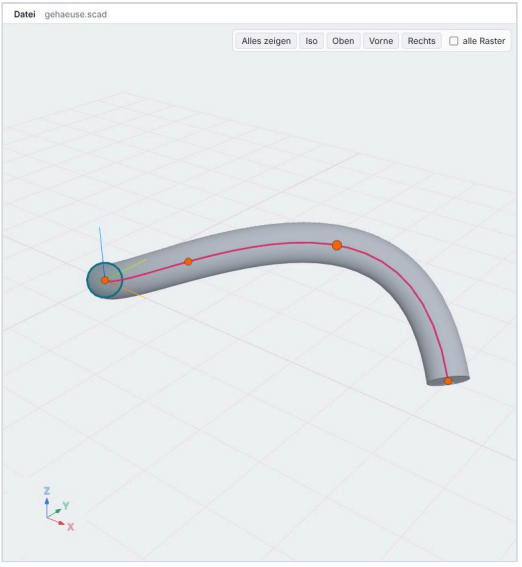
loft_sweep.scad, Cursor auf loft(): Ergebnis halbtransparent, alle drei Profile (Kreis unten, Quadrat auf z = 15, Kreis auf z = 30) als Umrisse an ihrer Lage.

C12 sweep

VORHER



DRITTEN PFADPUNKT GEZOGEN



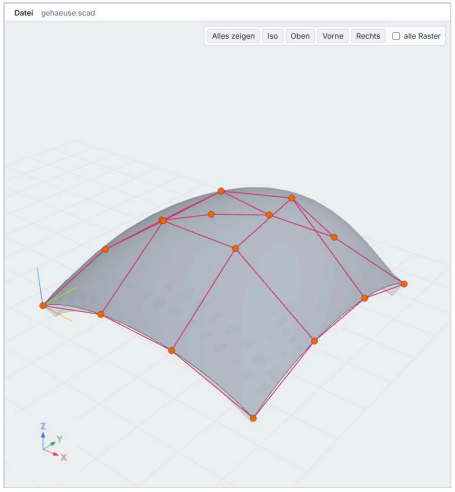
CODE

```
sweep(path = [[0, 0, 0], [20, 0, 10], [40, 10, 10], [60, 10, 0]], interp = "spline")
    circle(3);
// nach dem Ziehen:
sweep(path = [[0, 0, 0], [20, 0, 10], [39, 12, 14], [60, 10, 0]], interp = "spline")
```

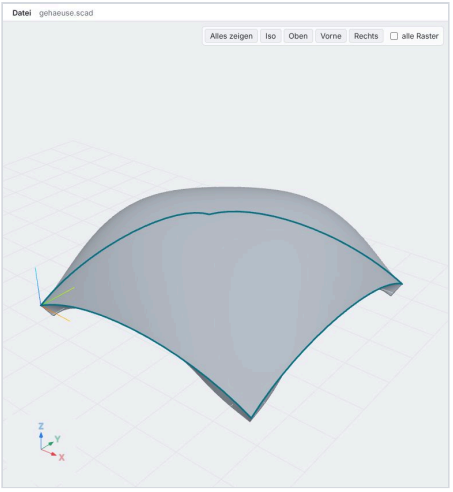
Cursor auf sweep: Pfad als Kurve (kubischer Spline durch die Punkte wie im Kernel, Sehnenlängen-Parameter), ein Griff an jedem Literal-Punkt, das Profil (Kreis) als Umriss am Startpunkt senkrecht zur Starttangente. Den dritten Punkt mit Alt frei gezogen: nur dieses Literal ändert sich, Kurve und Körper folgen.

C13 surface und thicken

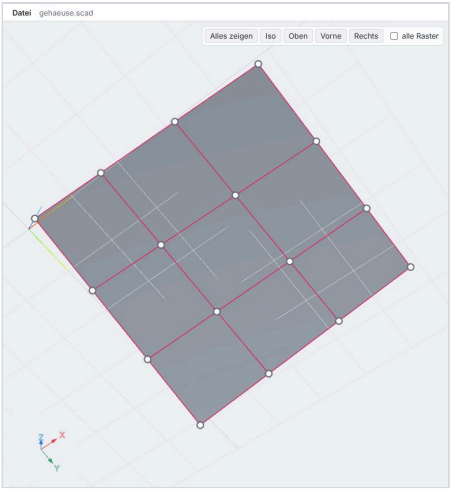
CURSOR AUF SURFACE: KONTROLLNETZ



CURSOR AUF THICKEN(2)



OFFENE FLÄCHE VON UNTEN



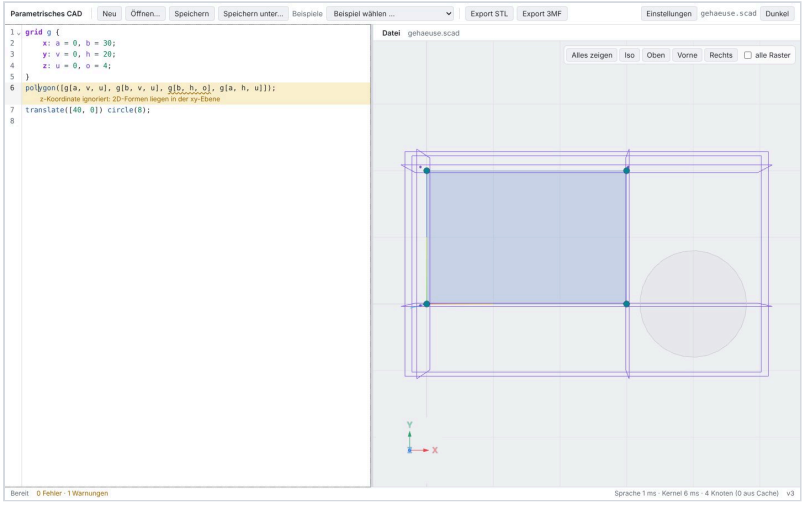
CODE

```
thicken(2)
    surface(points = [
        [[0, 0, 0], [10, 0, 3], [20, 0, 3], [30, 0, 0]],
        [[0, 10, 3], [10, 10, 10], [20, 10, 10], [30, 10, 3]],
        [[0, 20, 3], [10, 20, 10], [20, 20, 12], [30, 20, 3]],
        [[0, 30, 0], [10, 30, 3], [20, 30, 3], [30, 30, 0]]
    ]);
// nach dem Ziehen (Zeile 4):
        [[0, 10, 3], [9, 11, 12], [20, 10, 10], [30, 10, 3]],
// Fläche allein:
surface(points = [for (i = [0:3]) [for (j = [0:3]) [i * 8, j * 8, (i - 1.5) * (j - 1.5)]]]);
```

Cursor auf surface (4 × 4): Kontrollnetz (4 Zeilen + 4 Spalten als Linien) mit einem Griff an jedem Kontrollpunkt. Einen inneren Punkt mit Alt nach oben gezogen: nur sein Literal ändert sich. Cursor auf thicken(2): Ergebnis voll, die Fläche als Umriss. Eine offene Fläche allein (berechnete Punkte, gesperrte Griffe) wird beidseitig gerendert: von oben und von unten sichtbar.

C14 2D-Formen allein

EDITOR UND ANSICHT VON OBEN



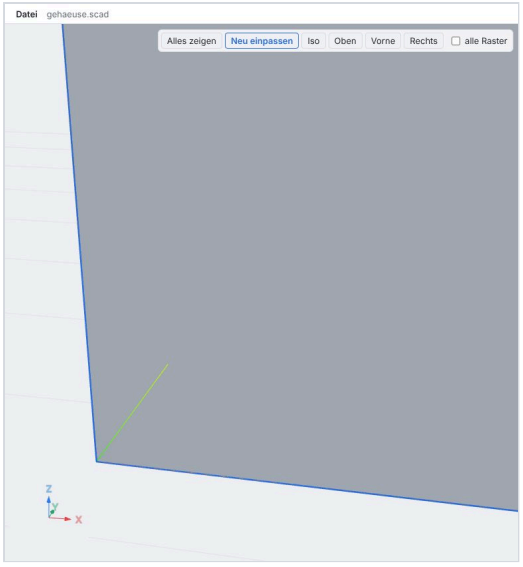
CODE

```
grid g {
    x: a = 0, b = 30;
    y: v = 0, h = 20;
    z: u = 0, o = 4;
}
polygon([g[a, v, u], g[b, v, u], g[b, h,
o], g[a, h, u]]);
translate([40, 0]) circle(8);
```

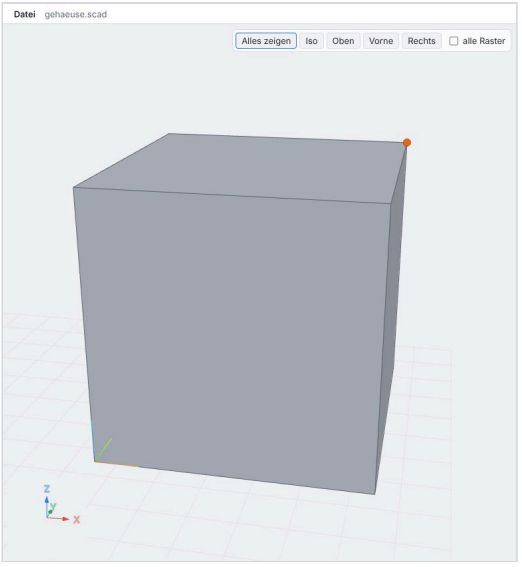
Nur 2D-Formen: Umriss mit leichter Füllung in der xy-Ebene (Ansicht von oben). Der Rasterpunkt g[b, h, o] hat z = 4: Zeile 6 gelb mit „z-Koordinate ignoriert: 2D-Formen liegen in der xy-Ebene“ am Zeilenende.

C15 Ansicht neu einpassen

KAMERA BEWEGT, MODELL 5× GRÖSSER: KNOPF



NACH „NEU EINPASSEN“



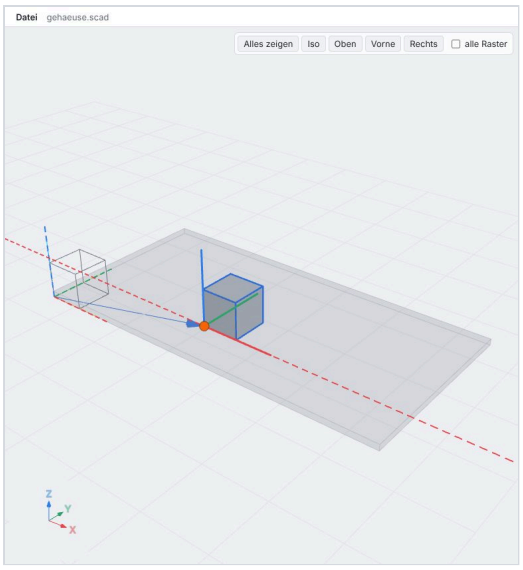
CODE

```
cube(10);  →  cube(60);  →  cube(62);  →  (Drehen)  →  cube(300);
```

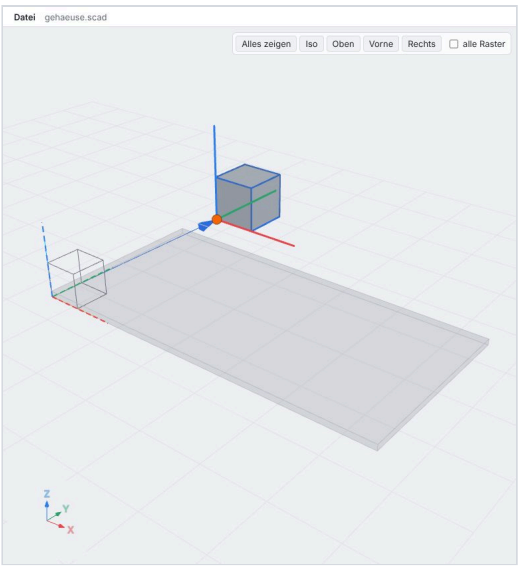
Solange der Nutzer die Kamera nicht bewegt hat, wird nach einer deutlichen Größenänderung (Diagonale × 1,6 oder Mitte um mehr als die halbe Diagonale verschoben) neu eingepasst: cube(10) → cube(60), Kameraabstand 32 → 194 mm; cube(62) ändert nichts. Nach dem Drehen mit der Maus bleibt die Kamera bei cube(300) stehen, oben erscheint „Neu einpassen“; ein Klick rahmt das Modell ein.

C16 Achsgebundenes Ziehen

UMSCHALTTASTE: ACHSE X AUS DER BEWEGUNG



ERST FREI, DANN TASTE Z



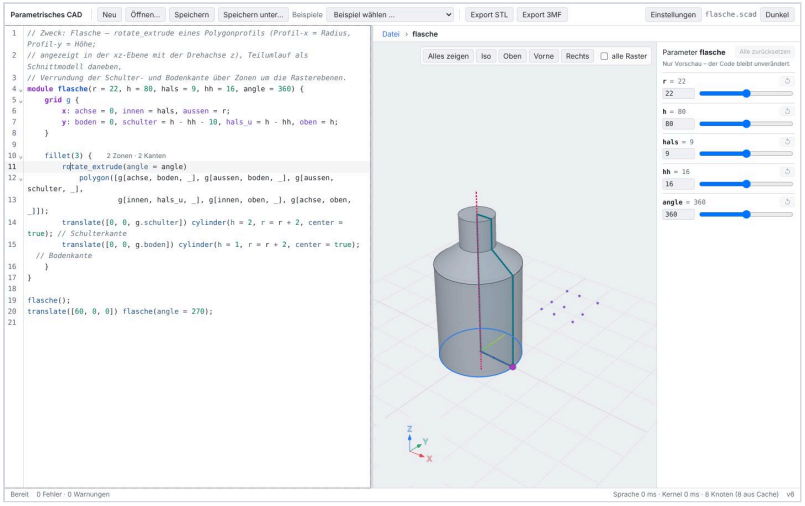
CODE

```
cube([60, 30, 1]);
translate([10, 5.5, 2.25]) cube(6);
// Umschalt + Ziehen (schräg, überwiegend x):
translate([27, 5.5, 2.25]) cube(6);
// erst frei, dann Taste Z:
translate([29, 7, 19]) cube(6);
```

Mit gedrückter Umschalttaste bestimmt die Mausbewegung die Achse (die Achse des lokalen Systems, deren Bildschirmrichtung am besten zur Bewegung passt); der Griff läuft auf einer Linie in Achsfarbe, nur diese Komponente ändert sich – y = 5.5 und z = 2.25 bleiben als Text erhalten (cad-eval schreibt unveränderte Literale nicht neu). Die Tasten X/Y/Z wählen die Achse direkt (erneut drücken: frei), auch mitten im Ziehen: die Achslinie geht dann durch die aktuelle Lage, die übrigen Komponenten behalten ihren Wert; die Tasten erreichen den Editor nicht.

C17-flasche Beispiel flasche.scad

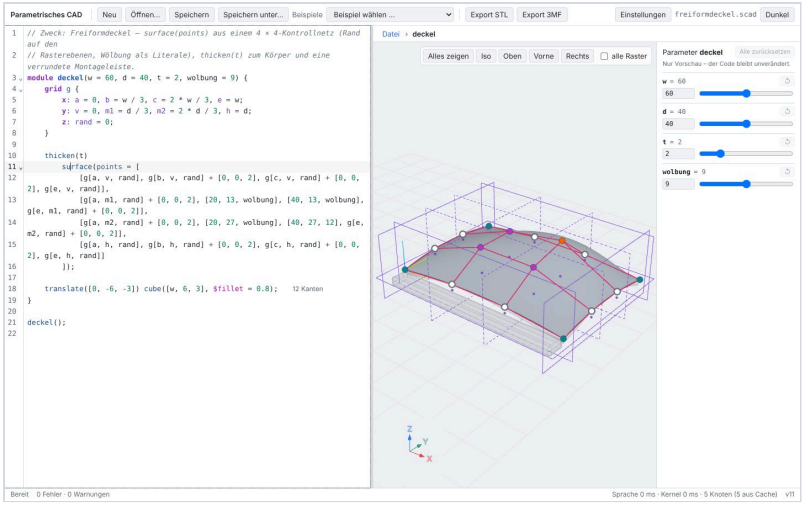
CURSOR IN ZEILE 11



rotate_extrude eines Polygons aus Rasterpunkten, fillet(3) mit zwei Zonen, Teilumlauf 270°. Fehlerfrei ausgewertet; Cursor in Zeile 11.

C17-freiformdeckel Beispiel freiformdeckel.scad

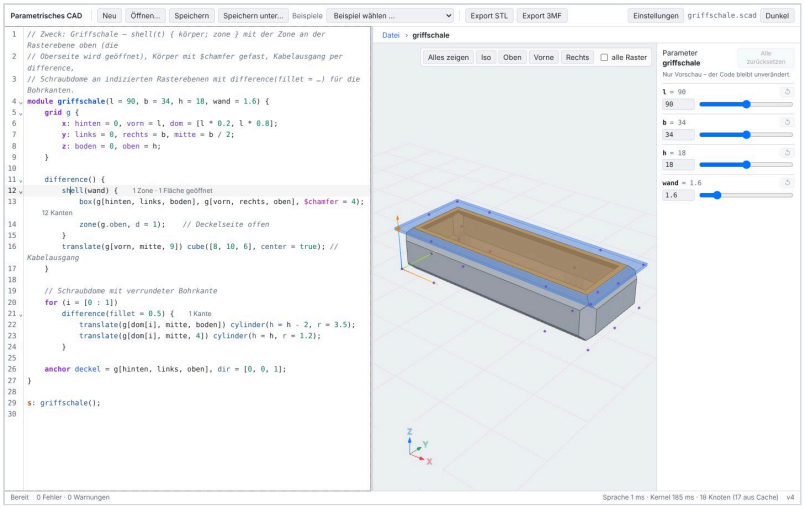
CURSOR IN ZEILE 11



surface aus einem 4 × 4-Netz (Rasterrand, Wölbung als Literale), thicken(2). Fehlerfrei ausgewertet; Cursor in Zeile 11.

C17-griffschale Beispiel griffschale.scad

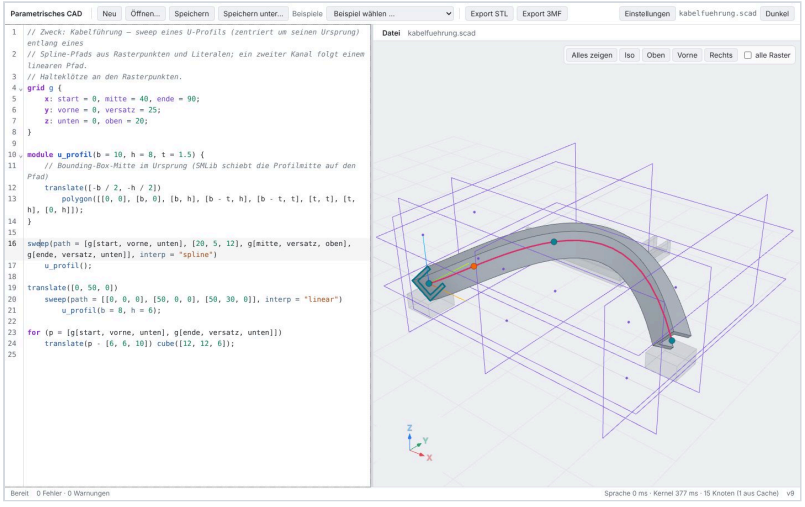
CURSOR IN ZEILE 12



shell mit Zone g.oben, Kabelausgang per difference, Schraubdome mit difference(fillet = 0.5). Fehlerfrei ausgewertet; Cursor in Zeile 12.

C17-kabelfuehrung Beispiel kabelfuehrung.scad

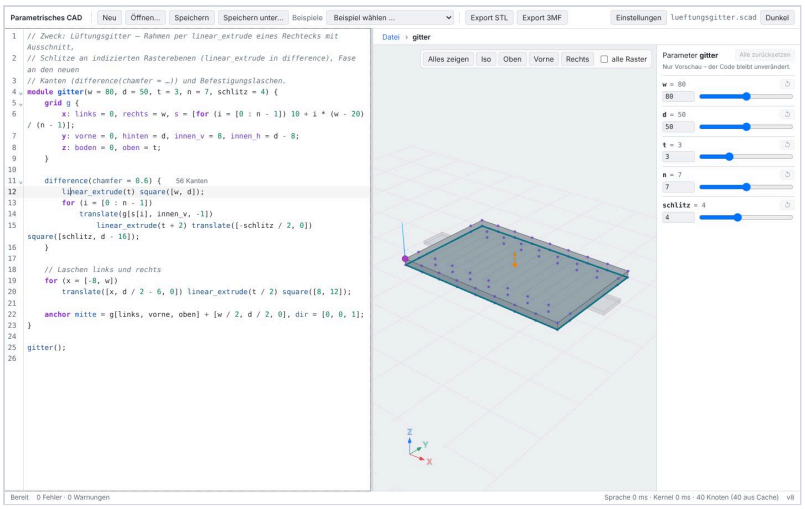
CURSOR IN ZEILE 16



sweep eines U-Profils entlang eines Spline-Pfads aus Rasterpunkten und Literalen, linearer Kanal. Fehlerfrei ausgewertet; Cursor in Zeile 16.

C17-lueftungsgitter Beispiel lueftungsgitter.scad

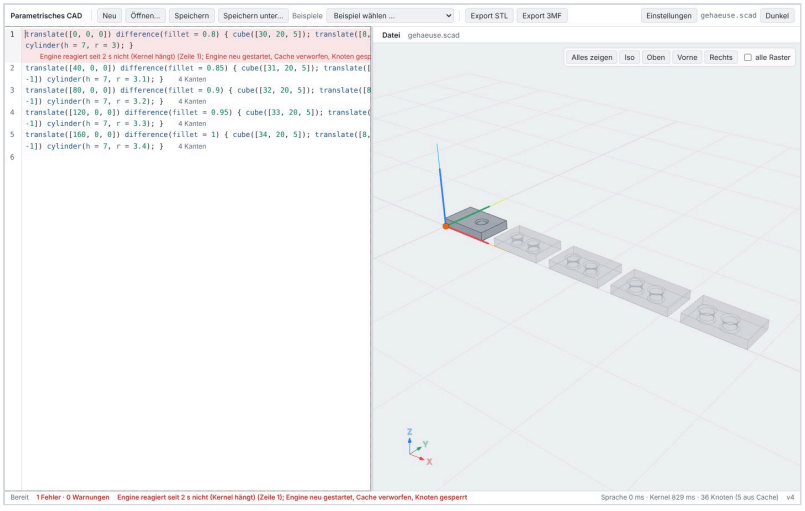
CURSOR IN ZEILE 12



linear_extrude für Rahmen und Schlitze an indizierten Rasterebenen, difference(chamfer = 0.6). Fehlerfrei ausgewertet; Cursor in Zeile 12.

C19 Watchdog-Sperre

NACH DEM NEUSTART: GESPERRTER KNOTEN ROT



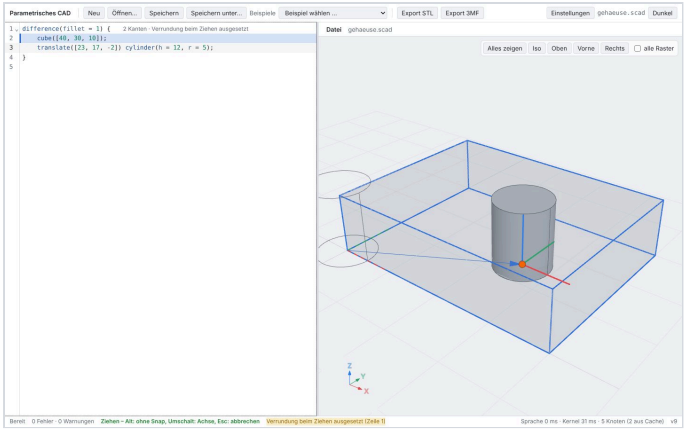
CODE

```
translate([0, 0, 0]) difference(fillet =
0.8) { cube([30, 20, 5]); translate([8,
10, -1]) cylinder(h = 7, r = 3);
translate([20, 10, -1]) cylinder(h = 7,
r = 3); }
// geändert:
translate([0, 0, 0]) difference(fillet =
0.7) { cube([34, 20, 5]); translate([8,
10, -1]) cylinder(h = 7, r = 2.8);
translate([20, 10, -1]) cylinder(h = 7,
r = 2.8); }
```

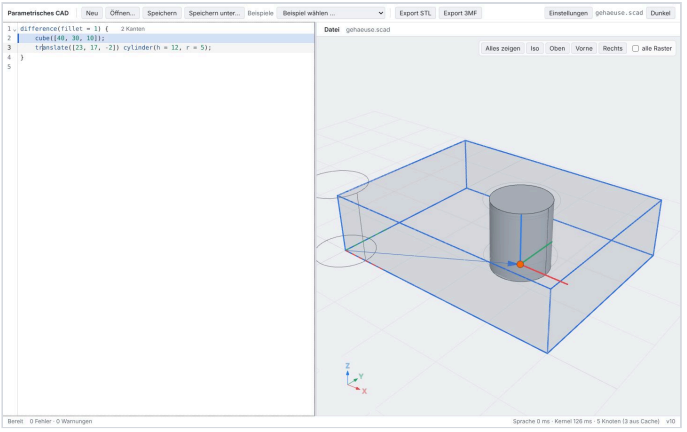
Watchdog auf 1,5 s gesetzt (?watchdog=1500), der Worker beim ersten Progress mit __debugHang 4 s blockiert: der Worker wird beendet und neu gestartet, der Schlüssel des Knotens aus Progress.current wird gesperrt; der neue Worker bekommt vor SetSource {type: "BlockKeys", keys: […]}. Der Knoten (translate) in Zeile 1 meldet „Zeitüberschreitung, Knoten gesperrt bis zur nächsten Änderung“ (Zeile rot), der Rest wird ausgewertet. Gesperrt wird der Inhaltsschlüssel: gleiche Knoten in anderen Zeilen wären mitgesperrt. Nach Ändern der Zeile wird sie normal berechnet; die Liste bleibt für weitere Neustarts erhalten (Unit-Test transport.test.ts).

C20 Entwurfsmodus beim Ziehen

WÄHREND DES ZIEHENS



NACH DEM LOSLASSEN



CODE

```
difference(fillet = 1) {
    cube([40, 30, 10]);
    translate([20, 15, -1]) cylinder(h = 12, r = 5);
}
// gezogen: translate([23, 17, -2]) cylinder(h = 12, r = 5);
```

Beim Ziehen des Zylinder-Griffs wird die Verrundung von difference(fillet = 1) ausgesetzt (Entwurfsmodus der Engine: Verrundungen, die länger als 25 ms brauchen): die Statuszeile zeigt „Verrundung beim Ziehen ausgesetzt (Zeile 1)“, am Zeilenende steht derselbe Hinweis (NodeReport.blend_suspended, im Szenenknoten als blend_suspended). Nach dem Loslassen wertet die Engine vollständig aus, Hinweis und Meldung verschwinden.